LMs Encode *What* a Sub-step Is and *When* It Ends with Distinct Mechanisms

Arnab Sen Sharma, Lisa Schut, Constanza Farro, Gabriele Sarti, Natalie Shapira, David Bau

As an LM writes code, how does it encode ***what*** sub-step it is on, and ***when*** it is done?

Checking if an integer is a perfect square

```
…
1. all of n’s digits are even
2. n is a perfect square
3. the sum of n's digits divides n
…
<code>
def check_conditions(n: int):
    a = [int(d) for d in str(n)]
    a = all(d%2 for d in a)
    b = int(abs(n)) ** 0.5
```

Sub-step boundary. Fetch `next()` instruction.

Clean: `b = b * b == abs(n)`

Patched:

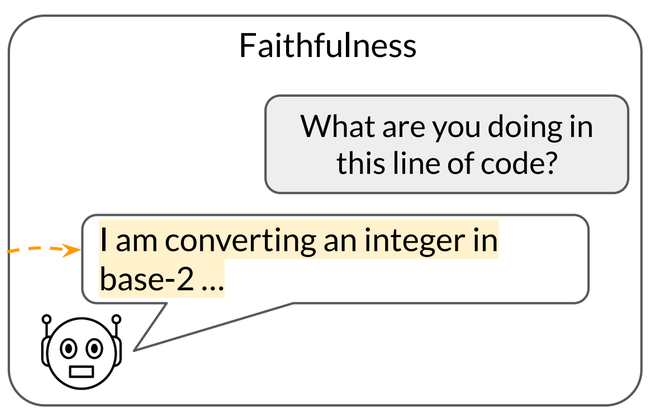
`c = sum(d for d in str(n))`

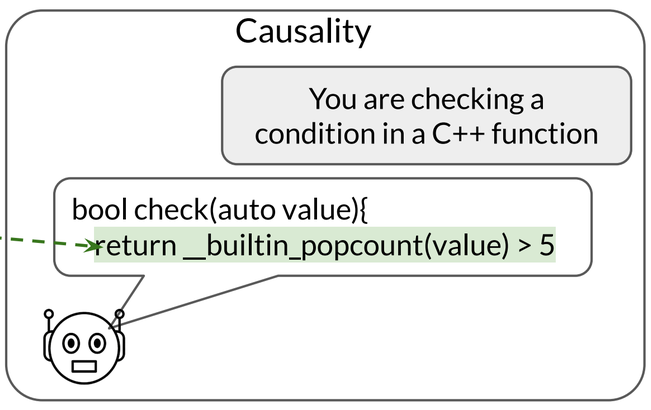
The sub-step is encoded well before it is realized in tokens.

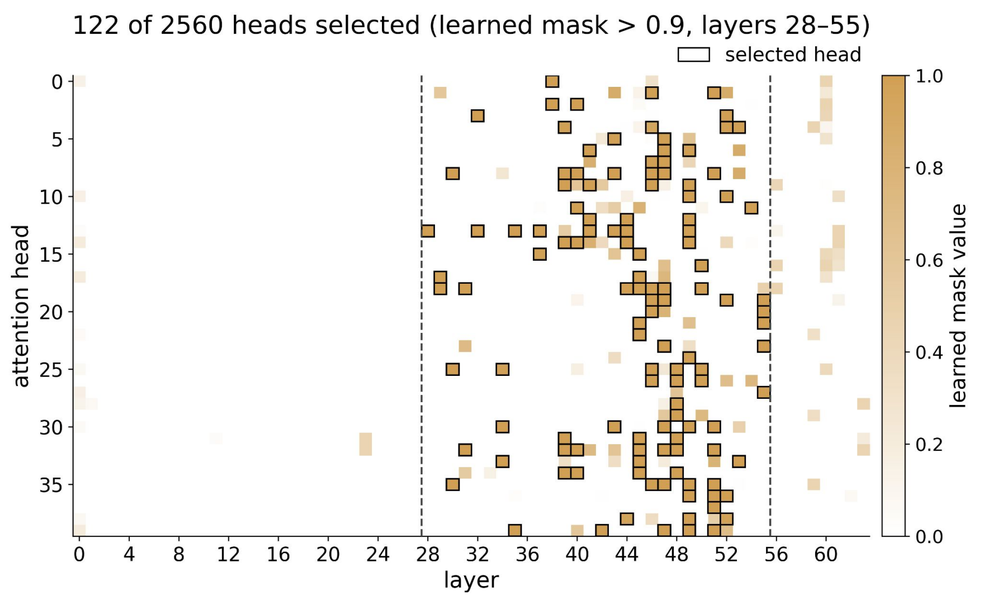
A sparse set of attn heads encodes the sub-step.

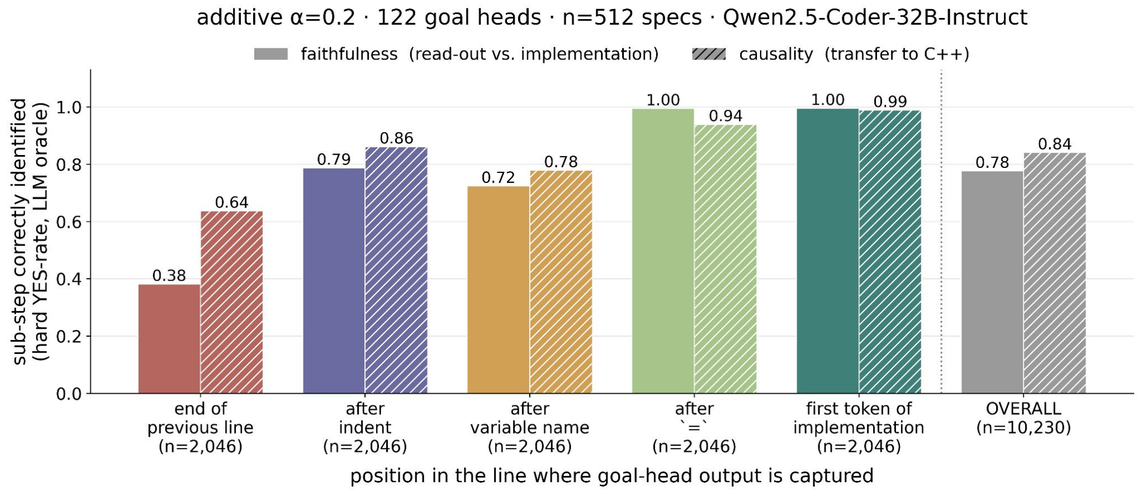
Steering with the aggregated head outputs in new context makes the LM **describe** or **implement** the sub-step.

```
…
1. n has more than 2 ones in
its binary representation
…
<code>
def check_conditions(n: int):
    a = bin(n).count(‘1’) > 2
    …
```









arnab-api.github.io/goalscope

Boundary encoding is not tied to the sub-step being implemented.

After a sub-step is done, a *different* set of attn heads marks the boundary and tells the LM to move on to the next sub-step.

```
…
1. n is coprime with 12
…
…
<code>
def check_conditions(n: int):
    a = math.gcd(n, 12) == 1
…
```

(i) cache from a boundary

```
…
1. the sum of n's proper divisors is greater than n
2. n is divisible by 16
3. n is a perfect cube
…
<code>
def check_conditions(n: int):
    sum_gt = sum(d for d in range(1, n) if n % d == 0)
    sub_gt = n > 0 and a > n
    is_div = n % 16
    is_div = is_div == 0
```

(ii) Patch to a non-boundary. Triggers next()

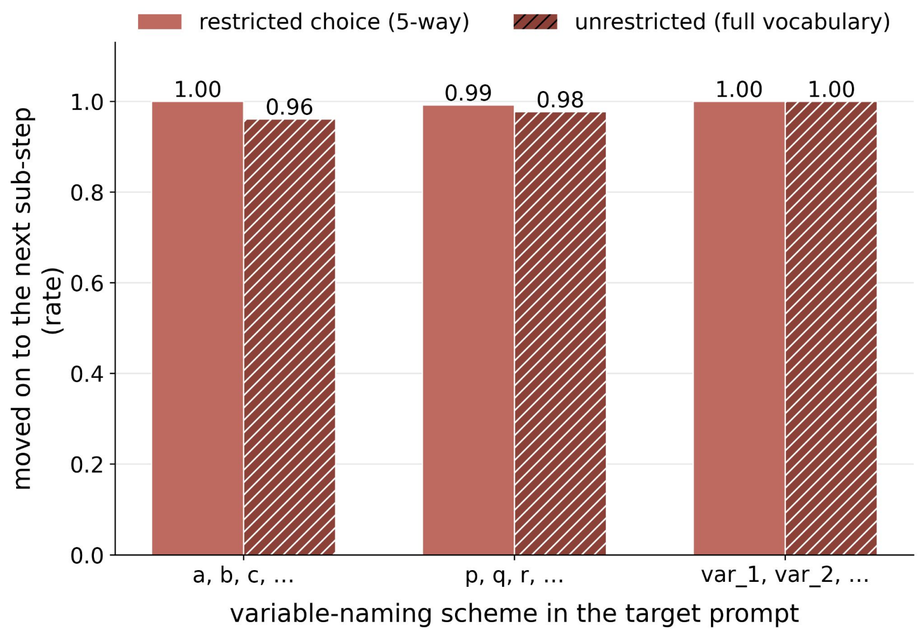
⇒ “ `is_cube = round(n**(1/3))**3 == n`”

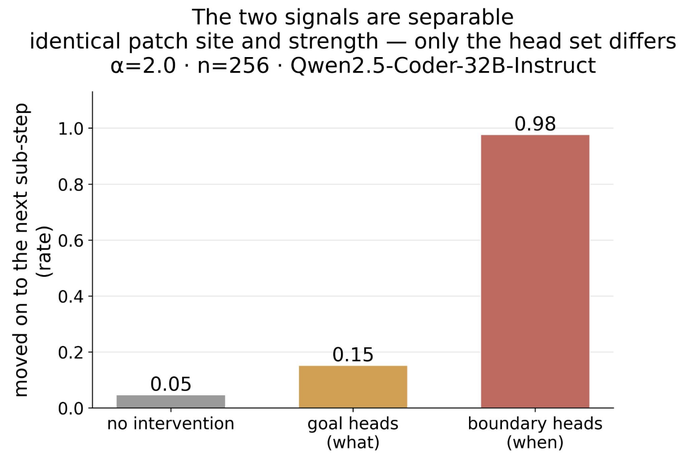
One move-on vector triggers the next sub-step under different variable names, in another language, even in a different traversal order.

```
…
…
 +  Check in REVERSE order of spec.
<code>
def check_conditions(n: int):
    is_cube = round(n**(1/3))**3 == n
    is_div = n % 16
    is_div = is_div == 0
```

Patch triggers the next step in the *reversed* order

⇒ “ `sum_gt = sum(d for d in …`”





Open questions: How do boundaries nest inside sub-sub-steps? Does the signal still fire when there are dependencies between sub-steps?